Israel Luiz de França Barberino

Arquitetura e Implementação de uma API REST para Gerenciamento de Usuários Utilizando Java, Spring Boot e Clean Architecture

São Paulo
2026

Israel Luiz de França Barberino

Arquitetura e Implementação de uma API REST para Gerenciamento de Usuários Utilizando Java, Spring Boot e Clean Architecture

Relatório técnico apresentado
como requisito parcial para
aprovação no Tech Challenge
– Fase 1 do curso de
Pós-Graduação em Arquitetura
e Desenvolvimento Java.

Faculdade de Informática e Administração Paulista – FIAP

São Paulo
2026

RESUMO

Este relatório técnico apresenta o projeto, a arquitetura e a implementação de uma API REST voltada ao gerenciamento de usuários, desenvolvida no contexto do Tech Challenge – Fase 1 do curso de Pós-Graduação em Arquitetura e Desenvolvimento Java da FIAP. A solução foi construída utilizando Java 17 e Spring Boot 3.x, fundamentada em princípios consolidados da engenharia de software, com destaque para a *Clean Architecture* e os princípios SOLID.

O sistema contempla operações completas de criação, consulta, atualização e remoção de usuários, bem como validação de credenciais de acesso, adotando práticas de segurança, validação em múltiplas camadas, documentação automatizada e execução reprodutível por meio de Docker Compose. O relatório enfatiza as decisões arquiteturais, os *trade-offs* envolvidos e os impactos dessas escolhas na manutenibilidade, testabilidade e evolução futura do sistema.

Palavras-chave: API REST. Clean Architecture. Spring Boot. Engenharia de Software. Docker.

Abstract

This technical report presents the design, architecture, and implementation of a REST API for user management, developed as part of the Tech Challenge – Phase 1 of the Graduate Program in Java Architecture and Development at FIAP. The solution was built using Java 17 and Spring Boot 3.x, grounded in established software engineering principles, particularly Clean Architecture and SOLID principles.

The system provides full CRUD operations, credential validation, multi-layer validation mechanisms, automated API documentation, and reproducible execution through Docker Compose. This report focuses not only on functional implementation but also on archi- tectural decisions, trade-offs, and their impact on maintainability, testability, and future system evolution.

Keywords: REST API. Clean Architecture. Spring Boot. Software Engineering.

Sumário

Sumário.....5
1. INTRODUÇÃO.....6
1.1. OBJETIVO GERAL.....6
1.2. OBJETIVOS ESPECÍFICOS.....6
2. METODOLOGIA.....7
2.1. PROCESSO DE DESENVOLVIMENTO.....7
2.2. FERRAMENTAS E TECNOLOGIAS.....7
2.2.1. Linguagem e Runtime.....7
2.2.2. Framework e Bibliotecas.....8
2.2.3. Persistência.....8
3. ARQUITETURA DA APLICAÇÃO.....9
3.1 CAMADAS DA APLICAÇÃO.....9
3.2. PADRÕES DE DESIGN IMPLEMENTADOS.....9
4. MODELAGEM DE DADOS.....10
4.1. OBJETO DE VALOR: ADDRESS.....10
5. IMPLEMENTAÇÃO E ESTRUTURA DO CÓDIGO.....11
5.1. ORGANIZAÇÃO DE PACOTES.....11
5.2 TRATAMENTO DE EXCEÇÕES.....11
6. DESIGN DOS ENDPOINTS REST.....11
6.1. ESPECIFICAÇÃO DOS ENDPOINTS.....11
7. DOCUMENTAÇÃO E TESTES.....12
7.1. DOCUMENTAÇÃO AUTOMATIZADA.....12
7.2. TESTES UNITÁRIOS E AUTOMATIZADOS.....12
8.1. EXECUÇÃO COM DOCKER COMPOSE.....13
8.2. SEGURANÇA DE DADOS.....13
10. CONCLUSÃO.....15
Referências.....16

1. INTRODUÇÃO

O desenvolvimento de sistemas distribuídos modernos tem ampliado significativamente a relevância de APIs REST como elemento central de integração entre aplicações. Em ambientes corporativos, decisões arquiteturais inadequadas em serviços backend tendem a gerar sistemas de difícil manutenção, alto acoplamento e elevada dívida técnica ao longo do tempo.

Nesse contexto, este trabalho aborda o problema arquitetural da construção de uma API REST que seja, simultaneamente, funcional, segura e arquiteturalmente sustentável. O foco não se restringe à entrega de *endpoints*, mas à adoção consciente de princípios arquiteturais que favoreçam a evolução do sistema.

1.1. OBJETIVO GERAL

Projetar e implementar uma API REST para gerenciamento de usuários, fundamentada em princípios consolidados de arquitetura de software, garantindo manutenibilidade, testabilidade e clareza estrutural.

1.2. OBJETIVOS ESPECÍFICOS

- Aplicar os princípios da *Clean Architecture* no desenvolvimento de uma API REST.
- Modelar entidades e relacionamentos de forma consistente com o domínio do problema.
- Projetar *endpoints* REST seguindo boas práticas de design e semântica HTTP.
- Implementar validações e mecanismos de segurança adequados.
- Disponibilizar documentação automatizada e ambiente de execução reprodutível.

2. METODOLOGIA

Este trabalho caracteriza-se como uma pesquisa aplicada, de natureza qualitativa, com abordagem prática e experimental. A metodologia adotada baseia-se no desenvolvimento incremental do sistema, com validação contínua por meio de testes automatizados e revisão arquitetural.
As decisões técnicas foram fundamentadas em literatura clássica da engenharia de software, aliadas à experimentação prática no contexto do projeto. A validação da solução ocorreu por meio da execução de testes unitários, de integração e de ponta a ponta, bem como pela análise da aderência aos requisitos funcionais e não funcionais definidos.

2.1. PROCESSO DE DESENVOLVIMENTO

O desenvolvimento seguiu um processo iterativo baseado nos seguintes passos:

1. **Análise de Requisitos**: Definição clara dos requisitos funcionais (CRUD, validação, busca) e não funcionais (segurança, testes, documentação).
2. **Design Arquitetural**: Seleção da *Clean Architecture* como padrão estrutural.
3. **Implementação Incremental**: Desenvolvimento iniciando pela camada de domínio.
4. **Testes Paralelos**: Desenvolvimento de testes em paralelo com a implementação.
5. **Documentação Contínua**: Atualização de documentos (Swagger, README) conforme evolução.
6. **Validação Final**: Verificação de conformidade e cobertura de testes.

2.2. FERRAMENTAS E TECNOLOGIAS

2.2.1. Linguagem e Runtime

- **Java 17**: Linguagem principal, utilizando recursos como *records* e *pattern matching*.
- **Maven 3.8+**: Gerenciador de dependências e automação de *build*.

2.2.2. Framework e Bibliotecas

- **Spring Boot 3.2.1**: Framework web e injeção de dependência.
- **Spring Data JPA**: Abstração de persistência com Hibernate.
- **Spring Security**: Autenticação e criptografia (BCrypt).
- **SpringDoc OpenAPI 2.1.0**: Geração de documentação Swagger.

2.2.3. Persistência

- **PostgreSQL 15**: Banco de dados relacional de produção.
- **H2 Database**: Banco em memória para testes isolados.

3. ARQUITETURA DA APLICAÇÃO

A arquitetura da aplicação foi concebida com base nos princípios da *Clean Architecture*, conforme proposto por Martin (2017). Essa abordagem promove a separação entre regras de negócio e detalhes de infraestrutura, favorecendo a independência de frameworks.

A aplicação é estruturada em camadas bem definidas. O fluxo de dependências ocorre sempre das camadas externas para as internas, respeitando o princípio da inversão de dependência.

3.1 CAMADAS DA APLICAÇÃO

1. **REST API Layer**: Controladores que expõem *endpoints* HTTP.

2. **Business Logic Layer**: Serviços que concentram a lógica de negócio.

3. **Data Access Layer**: Repositórios que abstraem operações de persistência.

4. **Domain Layer**: Entidades, *enums* e objetos de valor.

5. **Infrastructure**: Configurações de segurança, documentação e infra.

3.2. PADRÕES DE DESIGN IMPLEMENTADOS

- **Repository Pattern**: Abstração do acesso a dados.

- **Service Layer**: Concentração da lógica de negócio.

- **DTO Pattern**: Transferência de dados entre camadas.

- **Value Object**: Modelagem de Address como objeto de valor.

- **Global Exception Handling**: Tratamento centralizado de exceções (RFC 7807).

3.3. EXEMPLO DO MODELO DE ARQUITETURA ADOTADO:

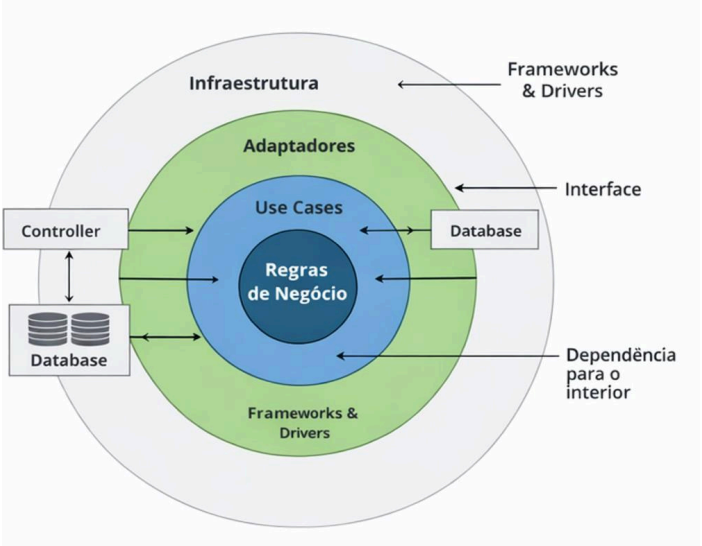

Figura 1 – Arquitetura em Camadas da Aplicação

4. MODELAGEM DE DADOS

O domínio do sistema é centrado na entidade *User*. A entidade possui os seguintes atributos principais: id (UUID), name, email, login, password, role, address, createdAt e updatedAt.

4.1. OBJETO DE VALOR: ADDRESS

O endereço foi modelado como um objeto de valor embutido (*Embedded*), visando maior coesão e simplicidade estrutural, evitando a complexidade de tabelas separadas para dados que são intrínsecos ao usuário.

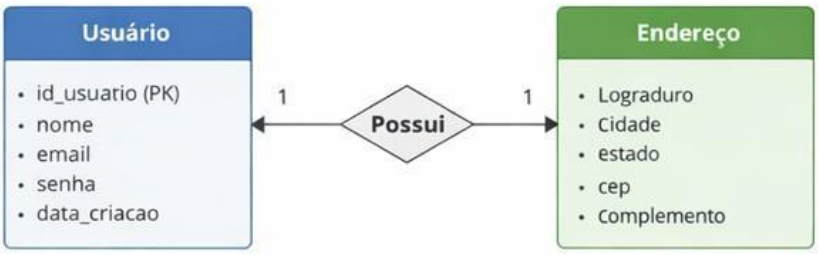

Figura 2 – Diagrama Entidade-Relacionamento (DER) do Sistema

5. IMPLEMENTAÇÃO E ESTRUTURA DO CÓDIGO

5.1. ORGANIZAÇÃO DE PACOTES

O projeto segue a estrutura de pacotes alinhada à *Clean Architecture*:

Listing 5.2 – Estrutura de Pacotes Java:

```
com . fiap . challenge /
        config /                    ( S e g u r a n a  e Open API)
        controller/                 ( Endpoints ( REST )
        service /                   Regras de ( n e g c i o )
        repository /                Interface de p e r s i s t n c i a ) d o
        entity / dto /              ( Modelo de m n i o )
        exception /                 ( Objetos de t r a n s f e r n c i a )
                                    ( Tratamento de erros)
```

5.2 TRATAMENTO DE EXCEÇÕES

O Global ExceptionHandler utiliza @RestControllerAdvice para capturar erros e retornar o padrão **RFC 7807**. Isso garante que o cliente da API receba mensagens claras sobre violações de regras de negócio ou erros de validação.

6. DESIGN DOS ENDPOINTS REST

A API adota versionamento explícito no *path* (/api/v1). Os *endpoints* foram projetados respeitando a semântica dos métodos HTTP.

6.1. ESPECIFICAÇÃO DOS ENDPOINTS

Método	**Endpoint**	**Descrição**
POST	/api/v1/auth/validate	Validar login e senha.
POST	/api/v1/users	Criar novos usuários.
GET	/api/v1/users/{id}	Busca usuário por ID.
GET	/api/v1/users?name=X	Busca usuários por nome (*case-insensitive*).
PUT	/api/v1/users/{id}	Atualizar dados cadastrais.

PATCH	/api/v1/users/{id}/password	Altera a senha.
DELETE	/api/v1/users/{id}	Remova o usuário.

Tabela 1 – Endpoints da API REST

7. DOCUMENTAÇÃO E TESTES

7.1. DOCUMENTAÇÃO AUTOMATIZADA

A interface gráfica **Swagger UI** está disponível para teste e visualização dos contratos. Ela permite que desenvolvedores consumam a API com exemplos reais de JSON.

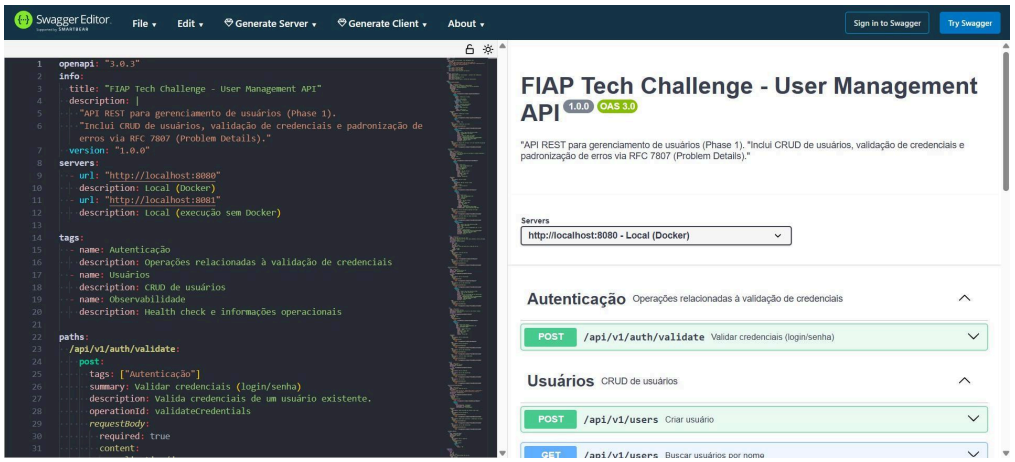

Figura 3 – Visualização do Swagger UI

7.2. TESTES UNITÁRIOS E AUTOMATIZADOS

O projeto implementou uma suíte com **44 casos de teste**, alcançando **100% de cobertura (JaCoCo Report)**. Foram utilizados JUnit 5 para a estrutura, Mockito para isolamento e MockMvc para testes de integração de controladores.

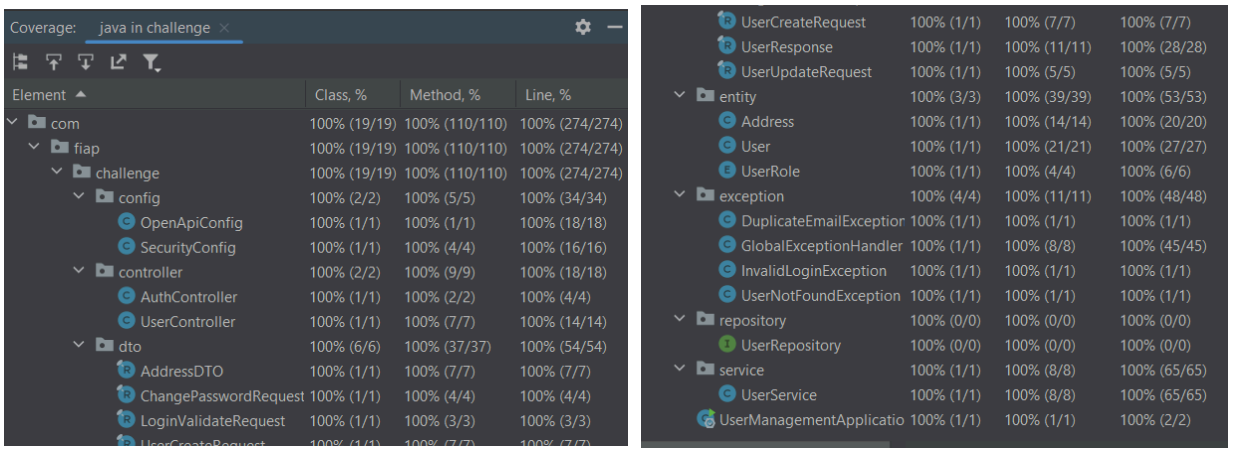

Figura 4 – Visualização da Coverage Window - IntelliJI

8. INFRAESTRUTURA E DEPLOYMENT

8.1. EXECUÇÃO COM DOCKER COMPOSE

A aplicação é conteinerizada, garantindo que o ambiente de banco de dados e a aplicação subam com um único comando: `docker-compose up -build`.

```yaml
services:
  postgres:
    image: postgres:15-alpine
    container_name: user_management_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: user_management
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: user_management_app
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/user_management
```

Figura 5 – Exemplo de docker-compose.yaml

8.2. SEGURANÇA DE DADOS

As senhas nunca são armazenadas em texto claro. Utiliza-se o algoritmo **BCrypt** com um fator de custo adequado, garantindo proteção contra ataques de dicionário e *rainbow tables*.

9. DISCUSSÃO

A escolha da *Clean Architecture* permitiu que a lógica de negócio ficasse isolada de preocupações como "qual banco de dados usar"ou "como o Spring Security funciona". Contudo, reconhece-se que para sistemas extremamente simples, essa estrutura pode ser considerada *over-engineering*. No contexto deste projeto, ela se justifica pela necessidade de escalabilidade e testabilidade rigorosa.

10. CONCLUSÃO

O projeto atende integralmente aos requisitos da Fase 1 do Tech Challenge. A API resultante é robusta, validada por uma alta cobertura de testes e documentada seguindo padrões de mercado. A arquitetura adotada fornece uma base sólida para a implementação de futuras funcionalidades, como autenticação JWT e autorização baseada em perfis (*roles*) de forma granular.

Referências

MARTIN, Robert C. *Clean architecture: a craftsman's guide to software structure and design*. Boston: Prentice Hall, 2017.

EVANS, Eric. *Domain-driven design: tackling complexity in the heart of software*. Boston: Addison-Wesley, 2003.

IETF. *RFC 7807: Problem Details for HTTP APIs*. 2016.

OWASP. *Password Storage Cheat Sheet*. Disponível em: <https://cheatsheetseries.owasp. org/>.